Fragmind

AI That Outgrows One Computer

Ultra-fast shared-memory IPC for distributing LLM inference and training across multiple machines with near-zero overhead.

167 ns	8.6 GB/s	96.93%	0%
message latency	tensor throughput	MNIST accuracy	overhead

The Problem

Modern AI models are enormous. GPT-4-class models have hundreds of billions of parameters that must all reside in memory during inference. A model like Llama 70B needs ~140 GB just to load. Llama 405B needs ~800 GB.

No single computer has enough memory. A high-end Mac Studio maxes out at 192 GB. The industry's current solution is NVIDIA H100 clusters costing millions of dollars. Most teams can't afford that.

The Solution

Split the AI's brain across multiple computers and have them work together as if they were one. Fragmind ("fragmented mind") takes a model that's too big for one machine and distributes its pieces across several machines that collaborate.

Fragmind-pigeon is the communication system that makes this work. It's the nervous system that lets the fragments talk to each other fast enough that the AI doesn't slow down.

How It Works

Imagine a factory assembly line: Person A installs the engine, Person B attaches the wheels, Person C paints the body. If the conveyor belt is slow, the whole factory slows down. Fragmind is the ultra-fast conveyor belt for AI:

- **Fragment A** (Computer 1) processes the first layers of the AI model
- **Fragment B** (Computer 2) processes the middle layers
- **Fragment C** (Computer 3) produces the final answer

The key: **shared memory**. Both programs read and write to the same physical memory location. No copying. It's like two people reading the same whiteboard instead of sending each other photocopies. For cross-machine, Fragmind uses **RDMA over Thunderbolt 5** — one computer reads directly from another's memory at 80 Gbps.

Performance: Fragmind vs. Alternatives

All numbers measured end-to-end on Apple M4 (ARM64). Fragmind vs. standard IPC:

Small Messages (latency, lower = better)

Transport	64 B	1 KB	Speedup vs TCP
Fragmind Ring	**167 ns**	**208 ns**	**61x / 51x**

Unix Socket (UDS)	3,125 ns	3,334 ns	3.3x / 3.2x
Pipe	2,959 ns	3,208 ns	3.5x / 3.3x
TCP (localhost)	10,208 ns	10,542 ns	1x (baseline)

Large Tensors (throughput MB/s, higher = better)

Transport	64 KB	1 MB	16 MB
Fragmind LOA	**8,806**	**8,607**	**6,060**
TCP (localhost)	2,686	5,469	5,136
Pipe	3,609	3,632	3,125
Unix Socket	1,678	2,501	2,429

At 16 MB (typical weight shard): Fragmind is **2.5x faster** than Unix sockets, **1.9x faster** than pipes, and **1.2x faster** than TCP.

Real-World Proof: Distributed MNIST Training

We trained a neural network on 60,000 handwritten digit images (MNIST), split across two workers communicating through Fragmind's shared memory pool:

Metric	Distributed (Fragmind)	Single-Process
Test accuracy	96.93%	97.00%
Avg batch latency	10.41 ms	10.43 ms
Training time (3 epochs)	29.3 s	29.4 s
Communication overhead	-0.2%	N/A
Data via shared memory	175.8 MB	0

The distributed model achieves **96.93% accuracy** — within 0.07% of the single-process baseline — with **zero measurable overhead**. The model learns correctly with activations and gradients flowing through shared memory.

Use Cases

Run a 70B Model on Two Macs

Two Mac Studios (192 GB each) connected via Thunderbolt 5. Mac 1 loads layers 0–39 (~70 GB), Mac 2 loads layers 40–79 (~70 GB). Fragmind handles activation transfer at 80 Gbps. Result: run a model that previously required datacenter hardware, using two desktop computers and a single cable.

Train on Your Team's Workstations

4 workstations, each owning a portion of the model. Training batches and gradients flow through Fragmind. No cloud GPU rental. No waiting weeks for compute time. Accuracy identical to single-machine training.

Mix Different Hardware

Put compute-heavy layers on your GPU machine, memory-heavy KV-cache on a high-RAM machine, data loading on fast storage. Each fragment specializes in what its hardware does best.

Why Not Existing Tools?

Approach	Limitation
NVIDIA NVLink	Only expensive NVIDIA GPUs ($30K+ per GPU)

gRPC / HTTP	Too slow for GB-scale tensor data
MPI / NCCL	Linux datacenter only, no Mac support
Bigger server	There's a ceiling — eventually no single machine is big enough

Fragmind is **free** (no special hardware), **fast** (167 ns latency, 8.6 GB/s), **cross-platform** (Mac + Linux), and **language-agnostic** (Go, Python, C).

Technical Summary

For technical audiences — what's inside Fragmind:

Component	Description
Ring (SPSC shm)	Lock-free shared-memory ring buffer. 167 ns write+read. ARM64-safe atomics.
LOA Pool	Zero-copy large-object arena. Alloc/commit/deref/release with atomic refcounting. 9 ns per cycle.
Pigeon Daemon	Per-host process: COI registry, LOA management, fragment attachment (SCM_RIGHTS), message r
COI Routing	Content-addressed routing by ConceptID + prefix bits. Fragments subscribe; pigeon delivers.
QUIC Gossip	Cross-host COI sync and message forwarding over QUIC (TLS 1.3).
RDMA Transport	Thunderbolt 5 RDMA via libibverbs. LOA pool registered as MR. Remote RDMA read at 80 Gb/s.
Python Bindings	Pure Python (mmap + struct). NumPy zero-copy tensor helpers. pip-installable.
C Header	All struct layouts with static_assert. Inline helpers for ring/LOA operations.

What Exists Today

- Pigeon daemon with LOA pool management and COI-based message routing
- Proven with real MNIST training: 96.93% accuracy, zero overhead
- Multi-process demo: 3 separate OS processes exchanging weight shards
- Python bindings with NumPy zero-copy tensor helpers (pip-installable)
- C header for integration with any language/framework
- RDMA over Thunderbolt 5 transport for cross-machine zero-copy
- 110 automated tests, comprehensive benchmark suite
- 9,610 lines of code across 56 source files